Visualização Geométrica, Rasterização e Recorte de Árvores Arteriais CCO

BCC327 – Computação Gráfica – Trabalho Prático 1

João Pedro Silva Cruz Pedro Vítor Silva Cruz

Departamento de Computação (DECOM)
Universidade Federal de Ouro Preto (UFOP)

11 de novembro de 2026

Roteiro da Apresentação

1. Contexto e Objetivos
2. Dados e Metodologia
3. Implementação
4. Resultados
5. Discussão e Conclusão

Contexto do Trabalho

Problema

Uma árvore arterial CCO é só uma lista de pontos, segmentos e um raio por vaso. Como transformar esses dados em imagem? O trabalho percorre o caminho completo: do arquivo VTK até os pixels.

- Modelos CCO fornecidos em formato `.vtk`, em 2D e em 3D.
- Cada vaso é um segmento, desenhado como linha ou como tubo.
- O atributo `raio` controla a cor e a espessura.
- Foco algorítmico: os algoritmos clássicos não são tratados como caixa-preta.

Objetivos

Objetivo Geral

Desenvolver uma aplicação que visualize, transforme, rasterize e recorte árvores arteriais CCO, implementando e comparando os algoritmos clássicos da disciplina.

1. Leitura, primitivas e atributos (raio $\rightarrow$ cor).
2. Transformações, observador e projeções.
3. Rasterização de linhas: Bresenham.
4. Rasterização de polígonos: *Scanline*.
5. Aliasing e antialiasing (*supersampling*).
6. Recorte de segmentos: Cohen–Sutherland, Liang–Barsky, Cyrus–Beck.
7. Recorte de polígonos: Sutherland–Hodgman.

Dados Utilizados

- Arquivos: `dados_modelos2D.zip` e `dados_modelos3D.zip`.
- Formato: `.vtk`.
- Estrutura:
  - pontos da geometria;
  - células de linha (segmentos);
  - atributo escalar `raio`.
- Uma célula pode ser uma polilinha de n pontos, ou seja, $n - 1$ segmentos.

Modelos analisados

	`tree2D_Nterm008`	`tree3D_Nterm016`
Pontos	16	32
Segmentos	15	31
Raio mínimo	0,2491	0,2261
Raio máximo	0,6668	0,6870
Dimensão	2D	3D

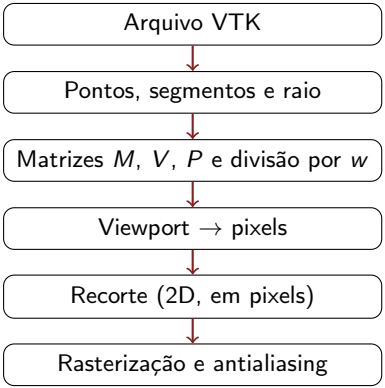
Tecnologias Utilizadas

- Linguagem: Python 3.14.3.
- Janela 3D e leitura do VTK: PyVista 0.49.2 / VTK 9.7.2 (OpenGL).
- Matemática e algoritmos: NumPy 2.4.4 e Python puro.
- Figuras: Matplotlib 3.10.8. Interface: PyQt5 5.15.11 + pyvistaqt 0.13.2.
- Entrada: arquivos `.vtk`. Saída: visualização interativa e figuras PNG.

Ambiente de execução

`pip install -r requirements.txt` e depois `python src/main_gui.py`. Os comandos completos estão no apêndice e no README.

Fluxo Geral da Aplicação



Etapa	**Arquivo**
Leitura	`modelo.py`
M, V, P	`transformacoes.py`
Viewport, Bresenham, *Scanline*, SSAA	`rasterizacao.py`
Recortes	`recorte.py`
Figuras e demos	`didatico.py`
Janela 3D	`visualizador.py`
Interface	`main_gui.py`

O que é do grupo e o que vem de biblioteca

Implementado pelo grupo

- Extração de segmentos e estatísticas.
- Matrizes M, V, P, NDC, viewport.
- Bresenham e DDA, *framebuffer*.
- *Scanline Fill* (ET e AET).
- *Supersampling* e erro quadrático médio.
- Cohen–Sutherland, Liang–Barsky, Cyrus–Beck.
- Sutherland–Hodgman.

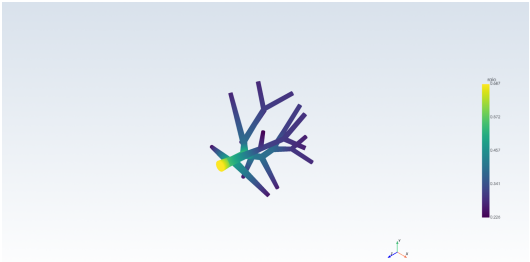
Com auxílio de bibliotecas

- PyVista/VTK: leitura do `.vtk`, janela OpenGL, tubos, sliders, antialiasing do VTK (F2).
- Matplotlib: figuras e mapa de cores.
- PyQt5: abas, botões e console.

Detalhe: a matriz M é nossa, mas é o VTK que a aplica ao desenhar; as nossas V e P valem na rasterização própria.

Representação Geométrica e Cores

- **Linhas** (espessura de tela fixa) ou **tubos** de 16 lados, com raio proporcional ao atributo.
- Raio por célula é interpolado para os pontos (exigência dos tubos).
- Rotação e escala em torno do centro do modelo.
- **Cor:** mapa contínuo *viridis*, com $t = \frac{r - r_{\min}}{r_{\max} - r_{\min}}$.
- Por quê: o raio é um escalar; o *viridis* é perceptualmente uniforme e legível em escala de cinza e para daltonismo.



Transformações, Câmera e Projeções

Pipeline próprio (NumPy)

$$\mathrm{p}_{\mathrm{clip}} = P\,V\,M\,\mathrm{p}, \qquad \mathrm{p}_{\mathrm{NDC}} = \frac{(x_c, y_c, z_c)}{w_c}$$

$$M = T(\mathrm{t})\,T(\mathrm{c})\,R_z R_y R_x\,S\,T(-\mathrm{c})$$

- V (tipo *lookAt*): mundo $\rightarrow$ câmera, que olha para $-z$.
- P: ortográfica (paralelepípedo) ou perspectiva (tronco de pirâmide, divisão por w).

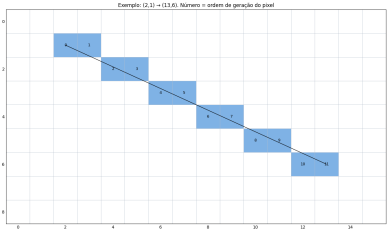
O que o usuário controla

- Translação, rotação e escala (sliders).
- Quatro vistas: frontal, lateral, topo e isométrica.
- Alternância ortográfica $\leftrightarrow$ perspectiva.
- Impressão de M, V e P no console.

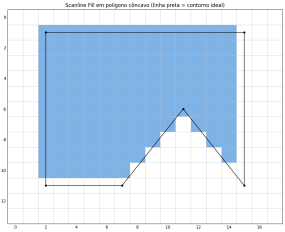
Mundo $\neq$ câmera: mudar de vista não move o objeto, só troca a matriz V.

Módulo 3: Bresenham

- Aritmética inteira, sem divisão nem ponto flutuante.
- $\Delta x = |x_1 - x_0|$, $\Delta y = -|y_1 - y_0|$, $e_0 = \Delta x + \Delta y$.
- A cada pixel, com $e_2 = 2e$:
  - $e_2 \geq \Delta y$: $e \mathrel{+}= \Delta y$ e avança em x;
  - $e_2 \leq \Delta x$: $e \mathrel{+}= \Delta x$ e avança em y.
- Forma simétrica: vale nos **8 octantes**.
- Exemplo $(2, 1) \to (13, 6)$: 12 pixels $= \max(|\Delta x|, |\Delta y|) + 1$.
- Teste: 1 500 segmentos aleatórios (extremos, nº de pixels, conectividade, desvio $\leq 0{,}5$).
- DDA difere em 273, sempre em empates de meio pixel.

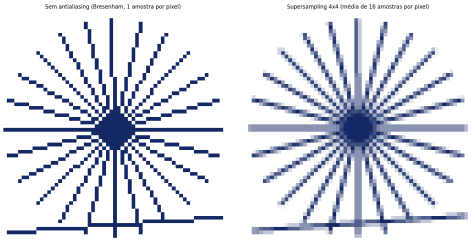


- **ET**: uma entrada por aresta não horizontal, $(y_{\min}, y_{\max}, x, \Delta x, \Delta y)$.
- **AET**: arestas que cruzam a linha atual; interseções ordenadas.
- Regra par-ímpar: trata polígonos côncavos.
- Convenções: topo exclusivo ($y_{\min} \leq y < y_{\max}$) e esquerda inclusiva, para vizinhos não se sobreporem.
- Interseções com frações exatas.
- Polígono de teste côncavo: 114 pixels (área geométrica 110).
- Teste: 300 polígonos aleatórios iguais a um teste independente de cruzamentos.



Módulo 5: Aliasing e Antialiasing

- Aliasing: "escada" de uma amostra por pixel.
- **Supersampling** $k \times k$ (próprio): desenha em resolução k vezes maior e reduz cada bloco pela média (filtro de caixa).
- A cor do pixel vira a fração de área coberta.
- Métrica: EQM contra uma referência 8×8.



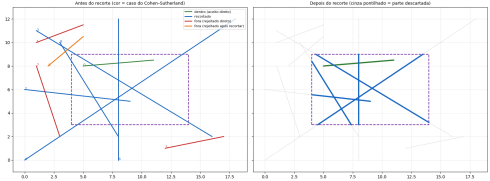
Cena de teste 64×64

	sem AA	SSAA 4×4
EQM	2414,0	54,1
cores distintas	2	17

Na janela 3D, a tecla F2 liga o SSAA do VTK.

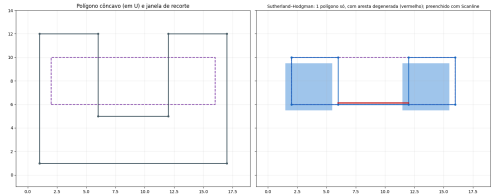
Módulo 6: Recorte de Segmentos

- **Cohen–Sutherland**: códigos de 4 bits; aceita ou rejeita de forma trivial; senão, recorta e repete.
- **Liang–Barsky**: forma paramétrica $P(u)$, com $p_k u \leq q_k$ por borda; no máximo uma divisão por borda.
- **Cyrus–Beck**: o mesmo raciocínio com normais; serve para janela *convexa* qualquer.
- Exemplo $(0,0) \to (12,10)$, janela $[2,10] \times [2,8]$: trecho visível (2,40; 2,00) $\to$ (9,60; 8,00).
- Cena de teste (10 segmentos): 1 aceito, 3 rejeitados, 5 recortados e 1 rejeitado só após recortar.
- Os três coincidem em 10 de 10; autoteste com 20.000



Módulo 7: Recorte de Polígonos

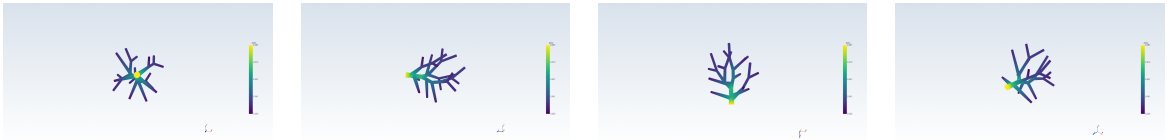
- **Sutherland–Hodgman**: uma borda da janela por vez; 4 casos por aresta (dentro–dentro, dentro–fora, fora–dentro, fora–fora).
- Implementado para janela convexa qualquer.
- Convexo: área 88 $\rightarrow$ 40.
- **Polígono em U** (côncavo): área 134 $\rightarrow$ 32, mas sai *um* polígono com um trecho degenerado $(6,6)-(12,6)$ em vez de dois retângulos.
- **Weiler–Atherton**: indicado quando há côncavos, várias regiões de saída ou janelas côncavas; percorre as listas de vértices de entrada e saída. Só discutido, não implementado.



Técnicas de Computação Gráfica no TP1

Técnica	**Como foi utilizada no projeto**
Recorte / *Clipping*	Cohen–Sutherland, Liang–Barsky e Cyrus–Beck (segmentos) e Sutherland–Hodgman (polígonos), todos próprios; Weiler–Atherton apenas discutido.
Rasterização / Antialiasing	Bresenham (com DDA para comparação), *Scanline Fill* e *supersampling* próprios; SSAA do VTK na janela 3D.
Cor / Atributos	Mapa contínuo *viridis* pelo raio; espessura pelo raio nos tubos.
Visibilidade, Iluminação, Ray Casting	Fora do escopo do TP1 (previstos para o TP2).
Transparência / Textura	Não usadas.

Observador e Projeções

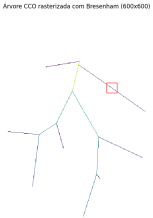


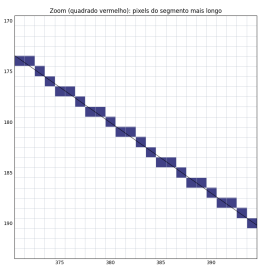
Projeção ortográfica (esq.) $\times$ perspectiva (dir.), mesma árvore e mesma vista

figures/projecoes.png

Na ortográfica o tamanho não depende da distância; na perspectiva, vasos distantes encolhem (noção de profundidade, mas comprimentos distorcidos).

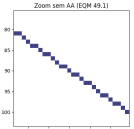
Árvore Rasterizada, com e sem Antialiasing

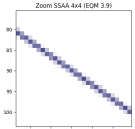






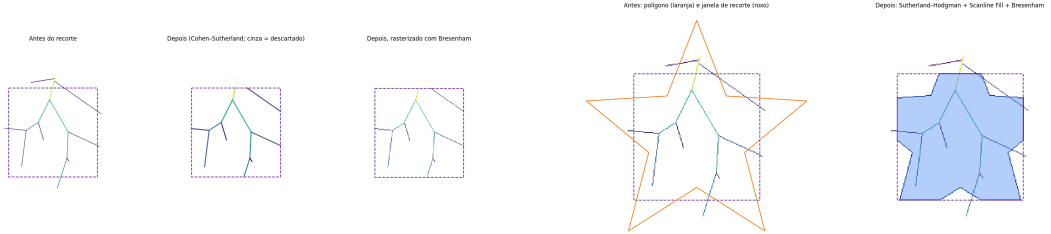






Modelo `tree2D_Nterm008`. Bresenham $\times$ DDA (600×600): 44 de 1 121 pixels mudam de posição. EQM (300×300) sem AA: 49,06; SSAA 4×4: 3,95; redução de 92%.

Recorte sobre a Árvore



Modelo `tree2D_Nterm008`, 300×300, janela 0,25–0,75 da imagem. Segmentos: 7 aceitos, 3 rejeitados, 5 recortados; Cohen–Sutherland: 6 interseções; Liang–Barsky: 57 divisões; os três idênticos em 15 de 15.

Validação: Autotestes

Algoritmo	**Teste (aleatório) e resultado**
Bresenham	1 500 segmentos: extremos, nº de pixels, conectividade em 8 vizinhos e desvio $\leq$ 0,5 pixel da reta ideal.
Scanline	300 polígonos estrelados: mesmos pixels de um teste independente de cruzamentos; vizinhos não se sobrepõem.
Recorte de segmentos	20 000 pares segmento/janela: os três algoritmos coincidem; trechos dentro da janela e sobre a reta original.
Sutherland–Hodgman	600 polígonos (côncavos incluídos) $\times$ janelas aleatórias: região resultante correta; janela disjunta dá vazio.

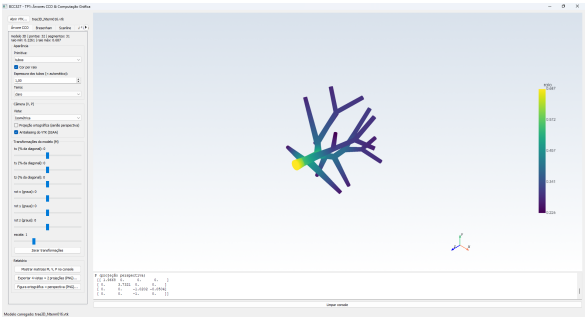
Comandos: `python src/rasterizacao.py` e `python src/recorte.py`.

Recurso Adicional: Interface Integrada

Recurso implementado

Interface PyQt5 que reúne todos os módulos em abas, com o 3D embutido.

- Abas: Árvore CCO, Bresenham, *Scanline*, Antialiasing, Recorte e Recorte de polígonos.
- Bresenham interativo: clique na grade para escolher os extremos.
- Tabelas e estatísticas no console embutido.
- Exportação das 4 vistas $\times$ 2 projeções e figura ortográfica $\times$ perspectiva.



Discussão

- **Conceitos aplicados:** pipeline M–V–P, rasterização, antialiasing e recorte de segmentos e de polígonos.
- **Dificuldades:** desempate de meio pixel (Bresenham $\times$ DDA); contas exatas no *Scanline*; arestas degeneradas no Sutherland–Hodgman; harmonizar a câmera do VTK com as nossas matrizes.
- **Biblioteca $\times$ grupo:** o VTK lê o arquivo, desenha em OpenGL, gera os tubos e monta V e P na janela 3D; os algoritmos pedidos são nossos.
- **Limitações:** sem *z-buffer*; *Scanline* com vértices inteiros; recorte só em 2D; Weiler–Atherton não implementado; o EQM mede a convergência para a referência 8×8, não o erro em relação à reta ideal.
- **Futuro (TP2):** visibilidade, iluminação, transparência, textura e ray casting.

Conclusões

- Os sete módulos foram implementados e demonstrados sobre árvores CCO.
- Cada algoritmo foi conferido contra uma verificação independente, em testes aleatórios.
- O caso do polígono em U mostrou na prática por que o Weiler–Atherton existe.
- A visualização CCO é um bom exemplo de visualização científica: o atributo do vaso guia cor, espessura e recorte.

Mensagem final

Do arquivo VTK até o pixel, os algoritmos de cada passo da imagem foram implementados e testados pelo grupo; a biblioteca entra na leitura, no desenho 3D e na janela interativa.

Referências

FOLEY, J. D. et al. *Computer Graphics: Principles and Practice*. 2. ed. Addison-Wesley, 1990.

SCHREINER, W.; BUXBAUM, P. Computer-optimization of vascular trees. *IEEE Transactions on Biomedical Engineering*, v. 40, n. 5, 1993.

KARCH, R. et al. A three-dimensional model for arterial tree representation, generated by constrained constructive optimization. *Computers in Biology and Medicine*, v. 29, n. 1, 1999.

BRESENHAM, J. E. Algorithm for computer control of a digital plotter. *IBM Systems Journal*, v. 4, n. 1, 1965.

LIANG, Y.-D.; BARSKY, B. A. A new concept and method for line clipping. *ACM Transactions on Graphics*, v. 3, n. 1, 1984.

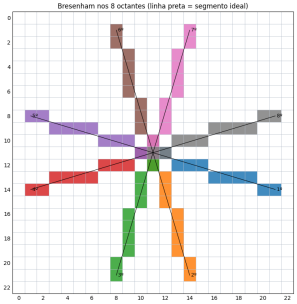
CYRUS, M.; BECK, J. Generalized two- and three-dimensional clipping. *Computers & Graphics*, v. 3, n. 1, 1978.

SUTHERLAND, I. E.; HODGMAN, G. W. Reentrant polygon clipping. *Communications of the ACM*, v. 17, n. 1, 1974.

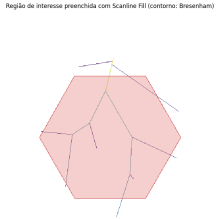
WEILER, K.; ATHERTON, P. Hidden surface removal using polygon area sorting. *ACM SIGGRAPH Computer Graphics*, v. 11, n. 2, 1977.

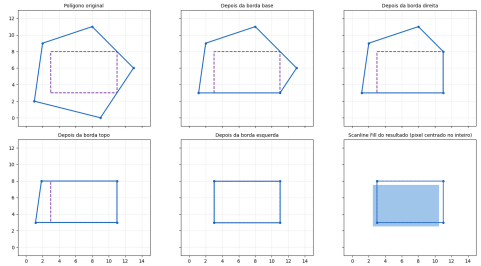
VTK Documentation. `https://vtk.org/`. PyVista: `https://pyvista.org/`.

Bresenham nos 8 octantes



Scanline e Sutherland–Hodgman: detalhes





Comandos de Execução

```
pip install -r requirements.txt
python src/main_gui.py                          # interface integrada
python src/visualizador.py ARQUIVO.vtk --info
python src/didatico.py bresenham --arvore ARQUIVO.vtk
python src/didatico.py antialiasing --arvore ARQUIVO.vtk --k 4 --ref 8
python src/didatico.py recorte --arvore ARQUIVO.vtk --janela 0.25 0.25 0.75
    0.75
python src/didatico.py recorte-poligonos --arvore ARQUIVO.vtk
python src/rasterizacao.py && python src/recorte.py   # autotestes
```

Observação

Modelos esperados em `data/dados_modelos2D/` e `data/dados_modelos3D/`. O README traz o roteiro completo de teste.

Obrigado!